Cours d'Informatique pour Débutants

Programmation Orientée Objet en Python & Conteneurisation avec Docker

Votre Enseignant

10 mars 2026

Table des matières

1 Programmation Orientée Objet en Python **3**
1.1 Introduction à la Programmation Orientée Objet (POO) . . . 3
1.2 Différence entre Programmation Procédurale et Orientée Objet . . . 3
1.3 Définition d'une classe en Python . . . 3
1.4 Création d'objets . . . 3
1.5 Attributs et Méthodes . . . 3
1.6 Le constructeur `__init__` . . . 4
1.7 Encapsulation . . . 4
1.8 Méthodes `get` et `set` . . . 4
1.9 Héritage . . . 5
1.10 Polymorphisme . . . 5
1.11 Exemple complet de programme Python . . . 5
1.12 Exercices Corrigés . . . 6
1.12.1 Exercice 1 : Livre . . . 6
1.12.2 Exercice 2 : Compteur . . . 7

2 Conteneurisation avec Docker **8**
2.1 Introduction à la conteneurisation . . . 8
2.2 Différence entre Machines Virtuelles et Conteneurs . . . 8
2.3 Présentation de Docker . . . 8
2.4 Installation de Docker . . . 8
2.5 Architecture de Docker . . . 8
2.6 Explication détaillée des commandes Docker . . . 8
2.6.1 `docker pull` . . . 8
2.6.2 `docker images` . . . 9
2.6.3 `docker run` . . . 9
2.6.4 `docker ps` . . . 9
2.6.5 `docker stop` . . . 9
2.6.6 `docker start` . . . 9
2.6.7 `docker rm` . . . 9
2.6.8 `docker exec` . . . 9
2.6.9 `docker build` . . . 10
2.7 Création d'un conteneur Docker étape par étape . . . 10
2.8 Création d'un Dockerfile . . . 10

2.8.1 Instructions détaillées 10
2.8.2 Exemple complet : Dockerfile pour une application Python 10
2.9 Exercices Pratiques avec Correction 11
2.9.1 Exercice 1 : Conteneuriser un script simple 11
2.9.2 Exercice 2 : Gestion de conteneurs 12

1 Programmation Orientée Objet en Python

1.1 Introduction à la Programmation Orientée Objet (POO)

La programmation orientée objet est un paradigme de programmation qui organise le code autour de la notion d'**objets**. Un objet est une entité qui possède des **attributs** (ses caractéristiques) et des **méthodes** (ses actions). Cette approche permet de modéliser le monde réel de manière plus intuitive.

1.2 Différence entre Programmation Procédurale et Orientée Objet

- **Procédurale** : On se concentre sur les fonctions et la logique. Les données et les fonctions sont séparées. (Exemple : C, Pascal).
- **Orientée Objet** : On regroupe les données (attributs) et les fonctions (méthodes) qui agissent sur ces données dans des **classes**. (Exemple : Python, Java, C++).

1.3 Définition d'une classe en Python

Une classe est un modèle (un moule) qui permet de créer des objets.

```python
class Chien:
    # Le corps de la classe
    pass
```

1.4 Création d'objets

Un objet est une **instance** d'une classe.

```python
class Chien:
    pass

# Creation d'objets (instances)
mon_chien = Chien()
ton_chien = Chien()

print(mon_chien) # Affiche : <__main__.Chien object at 0x...>
```

1.5 Attributs et Méthodes

- **Attributs** : Variables propres à l'objet.
- **Méthodes** : Fonctions propres à l'objet.

```python
class Chien:
    # Methode
    def aboyer(self):
        print("Woof !")

# Creation
mon_chien = Chien()
```

```python
mon_chien.nom = "Rex"  # Ajout dynamique d'un attribut (peu recommande)
mon_chien.aboyer()     # Appel de la methode
```

1.6 Le constructeur __init__

Le constructeur est une méthode spéciale appelée automatiquement à la création de l'objet. Il sert à initialiser les attributs.

```python
class Chien:
    def __init__(self, nom, age):
        self.nom = nom   # Attribut d'instance
        self.age = age   # Attribut d'instance

    def aboyer(self):
        return f"{self.nom} dit Woof !"

# Utilisation
mon_chien = Chien("Medor", 5)
print(mon_chien.aboyer())
```

1.7 Encapsulation

L'encapsulation est le principe qui consiste à restreindre l'accès direct à certaines données. En Python, on utilise une convention : un attribut précédé d'un underscore _ est considéré comme "protégé".

```python
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self._solde = solde  # Attribut protege

    def deposer(self, montant):
        if montant > 0:
            self._solde += montant

    def afficher_solde(self):
        return f"Solde : {self._solde} euros"
```

1.8 Méthodes get et set

On utilise des méthodes pour accéder (getter) et modifier (setter) les attributs protégés de manière contrôlée.

```python
class Personne:
    def __init__(self, nom):
        self._nom = nom

    # Getter
    def get_nom(self):
        return self._nom
```

```python

    # Setter
    def set_nom(self, nouveau_nom):
        if nouveau_nom:
            self._nom = nouveau_nom
        else:
            print("Erreur : nom invalide")

p = Personne("Alice")
print(p.get_nom())
p.set_nom("Bob")
```

1.9 Héritage

L'héritage permet de créer une nouvelle classe (classe fille) à partir d'une classe existante (classe mère). La fille hérite des attributs et méthodes de la mère.

```python
class Animal: # Classe mere
    def __init__(self, nom):
        self.nom = nom
    def parler(self):
        return "..."

class Chat(Animal): # Classe fille
    def parler(self):
        return f"{self.nom} dit Miaou !"

class Chien(Animal): # Classe fille
    def parler(self):
        return f"{self.nom} dit Woof !"

# Utilisation
animaux = [Chat("Felix"), Chien("Rex")]
for a in animaux:
    print(a.parler())
```

1.10 Polymorphisme

Le polymorphisme (du grec "plusieurs formes") permet à différentes classes d'utiliser une même méthode de manière spécifique. Dans l'exemple ci-dessus, la méthode `parler()` est polymorphique.

1.11 Exemple complet de programme Python

```python
# Definition d'une classe Vehicule
class Vehicule:
    def __init__(self, marque, modele, annee):
        self.marque = marque
        self.modele = modele
        self.annee = annee
```

```python
        self._kilometrage = 0

    def rouler(self, km):
        if km > 0:
            self._kilometrage += km
            print(f"Le vehicule a roule {km} km.")
        else:
            print("Distance invalide.")

    def get_kilometrage(self):
        return self._kilometrage

    def __str__(self):
        return f"{self.marque} {self.modele} ({self.annee})"

# Heritage : Voiture est une sorte de Vehicule
class Voiture(Vehicule):
    def __init__(self, marque, modele, annee, nb_portes):
        super().__init__(marque, modele, annee) # Appel du constructeur parent
        self.nb_portes = nb_portes

    def klaxonner(self):
        print("Tut tut !")

    def __str__(self):
        return super().__str__() + f" - {self.nb_portes} portes"

# Programme principal
if __name__ == "__main__":
    ma_voiture = Voiture("Toyota", "Yaris", 2020, 5)
    print(ma_voiture)
    ma_voiture.rouler(150)
    print(f"Kilometrage: {ma_voiture.get_kilometrage()} km")
    ma_voiture.klaxonner()
```

1.12 Exercices Corrigés

1.12.1 Exercice 1 : Livre

Créez une classe `Livre` avec les attributs `titre`, `auteur` et `annee`. Ajoutez une méthode `description()` qui retourne une chaîne formatée.

Correction :

```python
class Livre:
    def __init__(self, titre, auteur, annee):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
    def description(self):
        return f"'{self.titre}' par {self.auteur} ({self.annee})"
```

```python

livre = Livre("1984", "George Orwell", 1949)
print(livre.description())
```

1.12.2 Exercice 2 : Compteur

Créez une classe `Compteur` avec un attribut privé `_valeur`. Implémentez les méthodes `incrementer()` et `get_valeur()`. Empêchez la valeur de devenir négative.

Correction :

```python
class Compteur:
    def __init__(self):
        self._valeur = 0
    def incrementer(self, pas=1):
        if pas > 0:
            self._valeur += pas
    def get_valeur(self):
        return self._valeur

c = Compteur()
c.incrementer(5)
print(c.get_valeur())
```

2 Conteneurisation avec Docker

2.1 Introduction à la conteneurisation

La conteneurisation est une méthode de virtualisation légère qui permet d'empaqueter une application et ses dépendances dans un environnement isolé appelé **conteneur**. Cela garantit que l'application fonctionne de manière identique quel que soit l'environnement (développement, test, production).

2.2 Différence entre Machines Virtuelles et Conteneurs

- **Machine Virtuelle (VM)** : Inclut le système d'exploitation invité complet, ce qui la rend lourde (plusieurs Go) et lente à démarrer. L'hyperviseur virtualise le matériel.
- **Conteneur** : Partage le noyau du système d'exploitation hôte. Il est léger (quelques Mo), démarre en quelques secondes et consomme moins de ressources.

2.3 Présentation de Docker

Docker est la plateforme de conteneurisation la plus populaire. Elle permet de créer, déployer et exécuter des applications dans des conteneurs.

2.4 Installation de Docker

- **Windows/Mac** : Télécharger Docker Desktop sur `https://www.docker.com/products/docker-desktop/`
- **Linux (Ubuntu/Debian)** :

```
sudo apt update
sudo apt install docker.io
sudo systemctl start docker
sudo systemctl enable docker

```

2.5 Architecture de Docker

- **Docker Engine** : Le moteur qui construit et exécute les conteneurs.
- **Images** : Modèles en lecture seule (comme une classe en POO) utilisés pour créer des conteneurs.
- **Conteneurs** : Instances exécutables d'une image (comme un objet).
- **Registry** : Dépôt d'images (ex : Docker Hub).

2.6 Explication détaillée des commandes Docker

2.6.1 `docker pull`

Télécharge une image depuis un registry sans l'exécuter.

```
docker pull hello-world
docker pull python:3.9-slim  # Image Python officielle
```

2.6.2 `docker images`

Liste les images téléchargées sur votre machine.

```
docker images
```

2.6.3 `docker run`

Crée et démarre un conteneur à partir d'une image.

- `-d` : mode détaché (en arrière-plan)
- `-it` : mode interactif avec terminal
- `-name` : donne un nom au conteneur
- `-p 8080:80` : mappe le port 8080 de l'hôte au port 80 du conteneur

```
docker run hello-world
docker run -d -p 8080:80 --name mon_nginx nginx
docker run -it python:3.9-slim bash
```

2.6.4 `docker ps`

Liste les conteneurs en cours d'exécution. `docker ps -a` liste tous les conteneurs (même arrêtés).

```
docker ps
docker ps -a
```

2.6.5 `docker stop`

Arrête un ou plusieurs conteneurs en cours d'exécution.

```
docker stop mon_nginx
docker stop id_conteneur
```

2.6.6 `docker start`

Démarre un conteneur arrêté.

```
docker start mon_nginx
```

2.6.7 `docker rm`

Supprime un ou plusieurs conteneurs (ils doivent être arrêtés).

```
docker rm mon_nginx
docker rm id_conteneur
```

2.6.8 `docker exec`

Exécute une commande dans un conteneur en cours d'exécution.

```
docker exec -it mon_nginx bash  # Ouvre un shell bash dans le conteneur
docker exec mon_nginx ls -l     # Liste les fichiers sans entrer dans le shell
```

2.6.9 `docker build`

Construit une image à partir d'un fichier `Dockerfile`.

```
docker build -t mon_app:latest .
# -t donne un nom et un tag a l'image
# . indique le chemin du contexte de build (souvent le dossier courant)
```

2.7 Création d'un conteneur Docker étape par étape

1. Télécharger une image : `docker pull nginx`
2. Créer et démarrer un conteneur : `docker run -d -name site_web -p 8080:80 nginx`
3. Vérifier qu'il tourne : `docker ps`
4. Ouvrir un navigateur à l'adresse `http://localhost:8080`
5. Arrêter le conteneur : `docker stop site_web`
6. Le supprimer : `docker rm site_web`

2.8 Création d'un Dockerfile

Un `Dockerfile` est un fichier texte contenant les instructions pour construire une image Docker.

2.8.1 Instructions détaillées

— **FROM** : Spécifie l'image de base. `FROM python:3.9-slim`
— **WORKDIR** : Définit le répertoire de travail dans le conteneur. `WORKDIR /app`
— **COPY** : Copie des fichiers de l'hôte vers le conteneur. `COPY requirements.txt .`
— **RUN** : Exécute des commandes lors de la construction de l'image (ex : installation de dépendances). `RUN pip install -r requirements.txt`
— **CMD** : Commande exécutée par défaut quand le conteneur démarre. Il ne peut y en avoir qu'une. `CMD ["python", "app.py"]`
— **EXPOSE** : Indique le port sur lequel le conteneur écoute (informationnelle). `EXPOSE 5000`

2.8.2 Exemple complet : Dockerfile pour une application Python

Imaginons une application Python simple avec Flask.

Structure du projet :

```
mon_app/
    app.py
    requirements.txt
    Dockerfile
```

Contenu de `app.py` :

```python
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello, Docker !"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Contenu de `requirements.txt` :

```
Flask==2.2.3
```

Contenu du `Dockerfile` :

```dockerfile
# Utiliser une image Python officielle comme base
FROM python:3.9-slim

# Definir le repertoire de travail
WORKDIR /app

# Copier le fichier des dependances
COPY requirements.txt .

# Installer les dependances Python
RUN pip install --no-cache-dir -r requirements.txt

# Copier tout le code de l'application
COPY . .

# Indiquer le port utilise par l'application
EXPOSE 5000

# Commande pour lancer l'application
CMD ["python", "app.py"]
```

Construction et exécution :

```bash
docker build -t ma_flask_app .
docker run -d -p 5000:5000 --name mon_site ma_flask_app
# Aller sur http://localhost:5000
```

2.9 Exercices Pratiques avec Correction

2.9.1 Exercice 1 : Conteneuriser un script simple

Créez un script Python `hello.py` qui affiche "Bonjour le monde !". Écrivez un `Dockerfile` basé sur `python:alpine` qui copie le script et l'exécute avec `CMD`.

Correction :

```python
# hello.py
print("Bonjour le monde !")

```

```
# Dockerfile
FROM python:alpine
WORKDIR /app
COPY hello.py .
CMD ["python", "hello.py"]

# Construction et execution
docker build -t hello .
docker run hello
```

2.9.2 Exercice 2 : Gestion de conteneurs

1. Lancez un conteneur `nginx` nommé `web1` sur le port 8081.
2. Vérifiez qu'il tourne.
3. Ouvrez un shell à l'intérieur et modifiez la page d'accueil (fichier `/usr/share/nginx/html/index.htm`
4. Arrêtez et supprimez le conteneur.

Correction :

```
# 1.
docker run -d --name web1 -p 8081:80 nginx
# 2.
docker ps
# 3.
docker exec -it web1 bash
echo "<h1>Ma page modifiee</h1>" > /usr/share/nginx/html/index.html
exit
# 4.
docker stop web1
docker rm web1
```